

Difficulty and Contribution-Based Cooperative Coevolution for Large-Scale Optimization

Peilan Xu, Wenjian Luo^{1b}, *Senior Member, IEEE*, Xin Lin, Yatong Chang, *Graduate Student Member, IEEE*, and Ke Tang^{2b}, *Senior Member, IEEE*

Abstract—Cooperative coevolution (CC) is a paradigm equipped with the divide-and-conquer strategy for solving large-scale optimization problems (LSOPs). Currently, the computational resource allocation schemes of most CC could be divided into two categories, namely, equal allocation to all subproblems and preference allocation to the subproblems with a large contribution. However, the difficult subproblems are not carefully considered by the existing computational resource allocation schemes. For these subproblems, the investment of computational resources cannot quickly improve the fitness value, which leads to their small early contribution and being neglected. In this article, we comprehensively analyze the imbalanced nature of the subproblems from their difficulty and contribution in LSOPs. First, we propose a method to quantify the optimization difficulty of the problems during the evolution process, which considers both the difficulty of the fitness landscape and the behaviors of the optimization algorithm. Then, we propose a novel both difficulty and contribution-based CC framework, called DCCC, which encourages the allocation of the computational resources to more contributing and more difficult subproblems. DCCC is tested on the CEC'2010 and CEC'2013 large-scale optimization benchmarks, and is compared with several typical CC frameworks and state-of-the-art large-scale optimization algorithms. The experimental results demonstrate that DCCC is very competitive.

Index Terms—Cooperative coevolution (CC), fitness-distance correlation (FDC), large-scale optimization.

Manuscript received 7 November 2021; revised 19 January 2022, 20 April 2022, 4 July 2022, and 19 August 2022; accepted 23 August 2022. Date of publication 25 August 2022; date of current version 3 October 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 61573327; in part by the Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies under Grant 2022B1212010005; and in part by the Guangdong Provincial Key Laboratory under Grant 2020B121201001. (Corresponding authors: Wenjian Luo; Ke Tang.)

Peilan Xu and Xin Lin are with the Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies, the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen 518055, Guangdong, China, and also with the School of Computer Science and Technology, University of Science and Technology of China, Hefei 230027, Anhui, China (e-mail: xpl@mail.ustc.edu.cn; iskcal@mail.ustc.edu.cn).

Wenjian Luo and Yatong Chang are with the Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies, the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen 518055, Guangdong, China (e-mail: luowenjian@hit.edu.cn; 20s151150@stu.hit.edu.cn).

Ke Tang is with the Guangdong Provincial Key Laboratory of Brain-Inspired Intelligent Computation, the School of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518055, Guangdong, China (e-mail: tangk3@sustech.edu.cn).

This article has supplementary material provided by the authors and color versions of one or more figures available at <https://doi.org/10.1109/TEVC.2022.3201691>.

Digital Object Identifier 10.1109/TEVC.2022.3201691

I. INTRODUCTION

IN ENGINEERING design and science research, there are many optimization problems involving a considerable number of decision variables [1], [2], [3], known as large-scale optimization problems (LSOPs). Meta-heuristic algorithms, such as differential evolution (DE) [4], [5], evolution strategy (ES) [6], [7], and particle swarm optimization (PSO) [8], [9], have achieved exciting results on the complex black-box optimization problems. However, as the dimension of the problem increases, the search space expands exponentially, resulting in the rapid deterioration in the performance of the optimization algorithms [10].

Cooperative coevolution (CC) is an efficient method that absorbs the idea of divide-and-conquer to solve the large-scale optimization [11], [12], [13], [14]. The basic way of CC is to decompose an LSOP into several low-dimensional subproblems (also called components), and allocate the computational resources to the components. There has been a lot of work dedicated to the problem decomposition methods. Early methods, such as static grouping [11], [15] and random grouping [16], [17], made no allowance for the interaction relationship between variables. Later, a class of variable interaction detection methods based on perturbation, such as the family of differential grouping [18], [19], [20], [21], strived to minimize the interaction between components, and achieved satisfactory grouping results.

Computational resource allocation is another important step that affects the performance of the CC framework. The traditional CC framework allocates computational resources to each component equally, and optimizes the components in a round-robin fashion [11]. Omidvar *et al.* [22] first realized the imbalance nature of the components, and proposed contribution-based CC (CBCC), which allocates more computational resources to the components with a larger contribution. Not surprisingly, CBCC evoked the interest of researchers in the study of the contribution-aware computational resource allocation, such as CBCC3 [23], CCFR [24], FCRACC [25], DCCA [26], and bandit-based CC framework (BBCC) [27]. The evidence gathered from experimental results demonstrates the effectiveness of the contribution-aware computational resource allocation compared to equal allocation. However, this type of scheme blindly prefers the components with the large contribution, and does not care about the differences in the difficulty of the components. As a result, the difficult components are usually neglected because of their small early contribution, and their influence cannot be discovered because of the lack of allocation of the available computational resources.

To the best of our knowledge, only Luo *et al.* [28] analyzed the imbalance in the difficulty of the components, and proposed the difficulty-based CC (DBCC) for many-modal optimization. DBCC adopts fitness-distance correlation (FDC) [29] to quantify the difficulty of the fitness landscape of each component as a basis for allocating computational resources. DBCC only measures the difficulty of the fitness landscape of components at the start of an algorithm run, and then allocates the computational resources once. However, DBCC cannot identify the influence of components because it adopts a static allocation scheme.

So far, there is no computational resource allocation scheme that could comprehensively consider the imbalance in the difficulty and contribution of components. Moreover, there is also a lack of a suitable method for estimating the difficulty of the components. Therefore, we are motivated to design a novel computational resource allocation scheme for the CC framework, and introduce a new indicator to evaluate the problem difficulty. Overall, our contributions could be summarized as follows.

- 1) We present the way to quantify the problem difficulty during the evolution process, which consists of two parts. On the one hand, we use FDC to measure the difficulty of the local fitness landscape explored by the population. On the other hand, we apply the success probability to describe the behaviors of the optimization algorithm. Finally, we combine the above two measures to formulate an indicator of the problem difficulty.
- 2) We comprehensively analyze the imbalance in the difficulty and contribution of the components, and propose a novel difficulty and contribution-based CC framework (DCCC). DCCC first selects the components with a larger contribution, and then allocates unequal amounts of computational resources to them in the next cycle based on their difficulty, among which the more difficult the component is, the more computational resources are allocated.

DCCC is tested on the CEC'2010 and CEC'2013 large-scale optimization benchmark suites. Several typical CC frameworks and state-of-the-art large-scale optimization algorithms are used as compared algorithms to verify the performance of DCCC. The experimental results demonstrate that DCCC is very competitive.

The remainder of this article is organized as follows. Section II introduces the related work. Section III describes the proposed method of estimating the problem difficulty and the framework of DCCC. Section IV shows the experimental design and results. Section V first presents the experimental results of DCCC on heterogeneous LSOPs, and then discusses the computational resource allocation and ablation experiments. Finally, Section VI concludes this article and introduces future work.

II. RELATED WORK

A. Large-Scale Optimization Problems

In this article, without loss of generality, we assume that the LSOP to be solved is a minimization problem in the

following form:

$$\arg \min_{\mathbf{x}} F(\mathbf{x}), \mathbf{x} \in \Omega \quad (1)$$

where $\Omega = [\mathbf{l}, \mathbf{u}]^D \subset \mathbb{R}^D$ is the search space of the problem, D is the problem dimension, and $\mathbf{l}$ and $\mathbf{u}$ are the lower and upper bounds of the problems, respectively. According to the interaction relationship among the variables, LSOPs could be classified into separable problems and nonseparable problems. Here, the general definition of a k -nonseparable problem is described as follows.

Definition 1 [30]: The problem $F(\mathbf{x})$ is k -nonseparable, iff

$$\arg \min_{\mathbf{x}} F(\mathbf{x}) = \left(\arg \min_{\mathbf{x}_1} F_1(\mathbf{x}_1), \dots, \arg \min_{\mathbf{x}_k} F_k(\mathbf{x}_k) \right) \quad (2)$$

where $\{\mathbf{x}_1, \dots, \mathbf{x}_k\}$ is a partition of the vector $\mathbf{x}$, i.e., $\mathbf{x} = \mathbf{x}_1 \cup \dots \cup \mathbf{x}_k$, and $\mathbf{x}_1 \cap \dots \cap \mathbf{x}_k = \emptyset$. Here, it is emphasized that for any subset of vector, there is no further partition to satisfy (2).

$F(\mathbf{x})$ is a fully separable function if $k = D$, $F(\mathbf{x})$ is a fully nonseparable function if $k = 1$, and $F(\mathbf{x})$ is a partially separable function if $1 < k < D$. The additive separable problem is a special k -nonseparable function, which is defined as follows.

Definition 2 [18]: A k -nonseparable function $F(\mathbf{x})$ is additively separable, iff

$$F(\mathbf{x}) = \sum_{i=1}^k F_i(\mathbf{x}_i). \quad (3)$$

B. Cooperative Coevolution

CC [31] is a paradigm equipped with the divide-and-conquer strategy for solving LSOPs. The process of the CC framework is to decompose a problem into several subproblems (components) and solve them separately, which could be summarized into two steps, problem decomposition and computational resource allocation. Here, we focus on reviewing the methods of the computational resource allocation of CC frameworks.

Cooperative coevolutionary genetic algorithm (CCGA) [11] is the first CC framework. In CCGA, the components are optimized in a round-robin manner, that is, each component is allocated equal computational resources. With an in-depth understanding of the context of the optimization problems, some biased resource allocation schemes have been proposed [10]. These schemes believe that it is reasonable to prioritize the computational resource allocation to influential components.

The contribution is widely adopted as an indicator to determine the computational resource allocation for each component. CBCC [22], [23] is one of the representatives of adopting contributions as an indicator for biased computational resource allocation, and the computational resources are tended to the components with a larger contribution. There are two phases in CBCC, namely, the exploration phase and the exploitation phase. In the exploration phase, the same amount of computational resources are allocated to each component to obtain their initial contribution, and then the component with the largest contribution is optimized in the exploitation phase. The early CBCC [22], i.e., CBCC1 and CBCC2, used

the historically accumulated fitness improvements of the component as their contribution. Moreover, in the exploitation phase, CBCC1 allocates the preset computational resources to the component with the largest contribution, and CBCC2 optimizes this component until the optimal solution no longer improve. As the latest version of CBCC, CBCC3 [23] adopts the recent fitness improvements of the component as its contribution, and always chooses the component with the largest contribution to optimize in the current exploitation phase.

Yang *et al.* [24] proposed a CC framework called CCFR, in which the most contributing component is optimized for a cycle. Different from CBCC3, CCFR limits the evolution of the stagnant subpopulations, and the contribution of the component is defined as the average of the last contribution and the current fitness improvements. Ren *et al.* [25] proposed a fine-grained computational resource allocation for CC framework, called FCRACC. According to the historical contribution of the component, the contribution expectation of the component in the next iteration is estimated to determine the priority of the component. Jia *et al.* [26] proposed a distributed CC with an adaptive resource allocation for large-scale optimization, called DCCA, which allocates unequal amounts of processors for each component. The priority of a component is defined as the ratio of its historically accumulated fitness improvements to the number of processors used. Each time two components are selected to compare their priorities, and the low-priority component provides a processor to the high-priority component. Kazimipour *et al.* [27] proposed a BBCC, which regards the computational resource allocation as a dynamic multi-armed bandit problem, and predicts the future revenue of the components based on historical fitness improvements.

In the field of many-modal optimization, Luo *et al.* [28] proposed a CC framework based on the difficulty, called DBCC, which expects that each component could be solved within the preallocated computational resources. The computational resource allocation of DBCC is related to the difficulty of the components. The higher the difficulty of the component is, the more computational resources are allocated.

C. Fitness-Distance Correlation

FDC was proposed by Jones and Forrest [29], which is employed to measure the difficulty of the fitness landscape. The major steps to calculate the FDC value are as follows. First, a set of sample points $X = \{x_1, x_2, \dots, x_n\}$ is given, and the fitness value associated with X is $\mathcal{F} = \{f_1, f_2, \dots, f_n\}$. Then, the distance $\mathcal{D} = \{d_1, d_2, \dots, d_n\}$ between the global optimal solution and each individual is calculated. Finally, the FDC value is calculated as follows:

$$r(X) = \frac{\sum_{i=1}^n (f_i - \bar{f})(d_i - \bar{d})}{\sqrt{\sum_{i=1}^n (f_i - \bar{f})^2} \sqrt{\sum_{i=1}^n (d_i - \bar{d})^2}} \quad (4)$$

where $\bar{f}$ and $\bar{d}$ are the mean values of sets $\mathcal{F}$ and $\mathcal{D}$, respectively.

The investigation suggested that the problems could be roughly divided into three categories according to the FDC value with the known global optimal solution. For minimization problems, the strong negative correlation implies

that the problem has the feature of deception, the strong positive correlation indicates that the problem is simple, and the weak correlation represents that the problem is difficult. In practice, because of the lack of prior knowledge about the global optimal solution, we usually adopt the best solution in the sample point set X to approximate the global optimal solution to calculate the FDC value. In this case, the deceptiveness of the problem cannot be known because the location of the optimal solution is unknown. Therefore, the feature of deception is no longer considered, and the larger absolute value of the FDC coefficient represents the difficult fitness landscape, otherwise the simple fitness landscape.

There has been much work on how to use FDC to assist EAs. Tan *et al.* [32] proposed a DE with adaptive mutation strategy. The main idea of it is to adopt FDC and ruggedness of information entropy to describe the fitness landscape features, and then establish the connection between them and the mutation operators by training the random forest. Chu *et al.* [33] used FDC as one of the meta-features of the problems, found the relationship between meta-features and algorithm performance, and designed a recommendation framework for PSO algorithms. Li *et al.* [34] proposed a simplified FDC as the selection probability of the mutation operators of DE. The calculation of the simplified FDC is as follows. First, the individuals are sorted from near to far according to their distance to the optimal solution in the population. Then, the fitness value of each individual and its latter individual is compared in turn. Finally, the simplified FDC is calculated as $r = \theta/(n-1)$, where θ is the number of individuals inferior to latter, n is the population size. Sallam *et al.* [35] proposed a variant of DE for constrained optimization problems, which employs FDC to determine the operator adopted by the next generation in multioperators DE. Luo *et al.* [28] used FDC to estimate the difficulty of the subproblems as the basis of the computational resources allocation of the CC framework for many-modal optimization.

III. PROPOSED ALGORITHM

A. Motivation

The contribution-aware CC frameworks utilize the imbalance in the contribution to distinguish the influence of the components. The computational resource is preferentially allocated to the components with larger contribution. However, the imbalance in the difficulty of the component could interfere with the performance of the contribution-aware CC frameworks.

For example, it is assumed that the minimization function $F(\mathbf{x})$ is additive separable, including two components, a 50D sphere function F_{sphere} and a 25D Ackley's function F_{ackley} , as shown in the following:

$$F(\mathbf{x}) = F_{\text{sphere}}(\mathbf{x}_1) + F_{\text{ackley}}(\mathbf{x}_2) \quad \begin{cases} F_{\text{sphere}}(\mathbf{x}_1) = \sum_{i=1}^D x_i^2 \\ F_{\text{ackley}}(\mathbf{x}_2) = a - a \exp\left(-b \sqrt{\frac{1}{D} \sum_{i=1}^D x_i^2}\right) \\ \quad - \exp\left(\frac{1}{D} \sum_{i=1}^D \cos(2\pi x_i)\right) + e \end{cases} \quad (5)$$

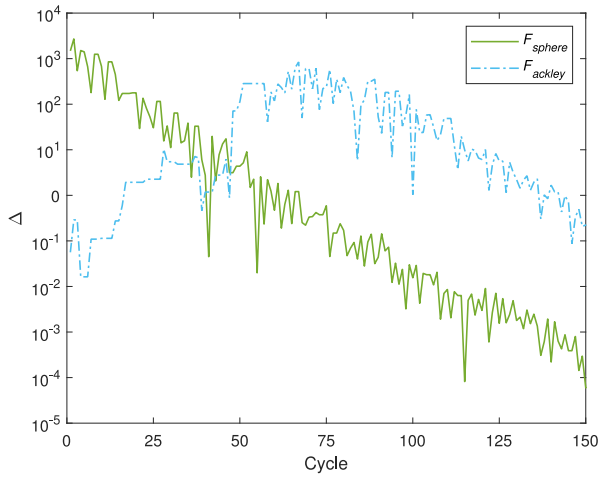


Fig. 1. Fitness improvement Δ of fitness values of components F_{sphere} and F_{ackley} in every cycle, where each cycle includes five iterations. In this case, the fitness improvement of F_{ackley} in the 4th cycle is for the first time greater than the fitness improvement of F_{sphere} in the 93rd cycle.

where $a = 10^4$, $b = 0.5$, global minimum $F(\mathbf{x}^*) = 0$ at $\mathbf{x}^* = (0, \dots, 0)$, and the boundary of each dimension is $[-32, 32]$. We adopt DE [36] to optimize these two components independently. The fitness improvements of two components in every cycle are separately recorded in Fig. 1.

It can be seen from Fig. 1 that the fitness improvement Δ corresponding to the component F_{sphere} decreases with the evolution process, because F_{sphere} is a simple unimodal function, which is easy to be solved by algorithms. Differently, the curve trend of Δ corresponding to the component F_{ackley} first rises and then falls, because F_{ackley} is a relatively complex multimodal function, and the optimization algorithm requires more computational resources to find the global optimal region. If the CC framework allocates computational resources only based on the contribution, the component F_{ackley} cannot obtain sufficient computational resources for optimization until F_{sphere} evolves 93 cycles. Under the premise of limited computational resources, this resource allocation scheme causes component F_{ackley} to be neglected, thereby weakening the performance of the CC framework. Generally, the population easily falls into the local optimum when solving difficult problems, then it could be rewarded if the investment of computational resources finds the global optimal region when the global to local optima contrast of the problem is large, such as the problems with the “funnel” landscape [37].

Fig. 1 indicates that the contribution-aware CC frameworks face two challenges in dealing with $F(\mathbf{x})$, namely, how to identify the influence of components more accurately and how to switch components more intelligently for optimization. Therefore, we consider combining the contribution with the difficulty of components in the design of the CC framework, and the proposed algorithm is called DCCC. Again, we use Fig. 1 as an example to explain the design idea of DCCC. Early in the evolution, DCCC invests computational resources to the component F_{sphere} with larger contributions to obtain large rewards quickly. As the contribution of F_{sphere} approaches F_{ackley} , DCCC optimizes components F_{sphere} and F_{ackley} simultaneously, and allocates

more computational resources to the more difficult component to identify its true influence. Finally, DCCC optimizes F_{ackley} if its contribution is larger. Compared with the contribution-aware CC frameworks, DCCC could identify the influence of difficult components earlier and allocate more computational resources to valuable difficult components, which is beneficial to the CC framework for solving this class of problems.

Moreover, we have the following two reasons to consider the difficulty of components when designing the CC framework. First, the ultimate goal of an optimization algorithm is to find the global optimal solution to the problem, and neglecting the optimization of any component may not achieve this goal. Particularly, without prior knowledge, any of the components should be assumed to be important. The contribution-aware CC framework relies only on the contribution of components to allocate computational resources, and difficult components risk being neglected because of their small early contribution. Second, the contribution-aware CC framework may keep optimizing a simple component that is almost solved, instead of optimizing a valuable difficult component, because its early contribution is small. As an example, in Fig. 1, F_{ackley} is more worthy of optimization than F_{sphere} in the middle of the evolution (e.g., after the 40th cycle). Because allocating computational resources to F_{ackley} improves the fitness value by more than 10^3 , and the cumulative fitness value improvement from continuously allocating computational resources to F_{sphere} is not more than 10^2 . However, the fitness improvements of F_{sphere} ($\Delta_{\text{sphere}} \approx 10^1$) is higher than F_{ackley} ($\Delta_{\text{ackley}} \approx 10^{-1}$) at the 40th cycle, which could mislead the contribution-aware CC framework into believing that F_{sphere} is more worthy of optimization than F_{ackley} .

In the remainder of this section, first, we present a method to dynamic estimate the problem difficulty. Then, we describe the details of DCCC.

B. Dynamic Estimation of the Problem Difficulty

In this article, the problem difficulty is determined by a combination of: 1) the difficulty of the fitness landscape and 2) the search behaviors of the optimization algorithm. First, without prior knowledge, or in a situation where the optimization behavior of the algorithm is approximate on both functions at a certain cycle, it could be predicted that in the next cycle, the more complex the fitness landscape is, the more difficult the optimization of the function. Second, as described by Mersmann *et al.* [38], the problem difficulty is also related to the behaviors of the algorithm, that is, the same problem is different in difficulty for different algorithms.

FDC could estimate the difficulty from the perspective of the fitness landscape of the problem, but there is an obstacle to employing FDC to assist in the design of the CC framework for large-scale optimization. The performance of FDC depends on the distribution and the size of the sample points. Especially for high-dimensional problems, a considerable number of sample points are required to improve the accuracy of estimating the global landscape.

The sample points set is the key to employing FDC to measure the difficulty of the fitness landscape. Although there is

the practice of selecting the initial population as the sample points [28], the method of the one-time measure exposes its major defect as the dimensions of the landscape increase, that is, sparse sample points are hard to estimate the difficulty of the global fitness landscape in the high-dimensional space. Consequently, we use the sample points during the evolution process to estimate the difficulty of the local fitness landscape to improve the performance of FDC, which is described as follows.

Given an initial population $P = \{x_1, x_2, \dots, x_n\}$ and the sample points set $\Lambda = \{X_1, X_2, \dots, X_n\}$, where $X_i = \{u_{t-\mathcal{J}+1}, u_{t-\mathcal{J}+2}, \dots, u_t\}$ is the set of the offspring of individual x_i during recent $\mathcal{J}$ iterations, the FDC coefficient of the population at t -th iteration is defined as follows:

$$r = \frac{1}{n} \sum_{i=1}^n |r(X_i)| \quad (6)$$

where n is the population size and $r(X_i)$ is the FDC coefficient corresponding to the set X_i , which is calculated according to (4).

The set of offspring in the previous cycle is employed as the sample points to estimate the difficulty of the local fitness landscape explored by this individual. Then, the collective behaviors of all individuals describe the difficulty of the local fitness landscape explored by the population, which is defined as the mean of the absolute value of the FDC coefficient of all individuals.

Schumer and Steiglitz [39] used the success probability to analyze the performance of (1 + 1)-ES, which is defined as the probability of the offspring replacing the parent. We extend this idea to a general optimization algorithm to quantify its search behaviors, which is described as follows.

Given a population P and its updated population, it is considered a successful update if the optimal solution in the population is improved. The search behaviors of the optimization algorithm are evaluated by the success probability during the recent $\mathcal{J}$ iterations, as shown in the following:

$$p_s = \frac{n_s}{\mathcal{J}} \quad (7)$$

where n_s is the number of success update within the recent $\mathcal{J}$ iterations.

Finally, we combine the correlation coefficient r and the success probability p_s to measure the problem difficulty, which is as follows:

$$d = 1 - p_s \cdot r. \quad (8)$$

The difficulty $d \in [0, 1]$ has a clear and intuitive physical explanation. That is, the weaker the correlation, the lower the success probability, and the higher the problem difficulty. Conversely, the stronger the correlation, the higher the success probability, and the lower the problem difficulty.

C. Proposed Cooperative Coevolution Framework

There is a phenomenon revealed by the example in Section III-A, that is, contribution-aware CC frameworks may avoid the difficult component when allocating computational resources, which results in the difficult component

being unable to obtain sufficient computational resources to be solved, even though it is influential to the whole. Therefore, both the difficulty and the contribution of the components should be simultaneously paid attention to when designing the CC framework. For this reason, we propose a novel DCCC.

DCCC first emphasizes the allocation of computational resources to more influential components, and the influence of a component is determined by its contribution δ . In DCCC, the contribution is defined as the magnitude of its fitness improvements to the overall fitness value, as shown in the following:

$$\delta_i = \lfloor \log_{10}(f'_{\text{best}} - f_{\text{best}}) \rfloor \quad (9)$$

where δ_i is the contribution of the i th component, f_{best} is the best fitness value after optimization, and f'_{best} is the best fitness value before optimization.

The components with the largest contribution δ_{max} are given priority to be allocated computational resources. Here, we add all components whose contribution is δ_{max} to the set Φ . then DCCC allocates unequal amounts of computational resources for each component based on their difficulty. The more difficult the component, the more computational resources are allocated. The number of iterations for each component C_i in the next cycle is calculated as follows:

$$\mathcal{J}_i = \begin{cases} \mathcal{J}_{\min} + \lfloor \alpha_i \mathcal{J}_{\text{inc}} \rfloor, & \text{if } \delta_i = \delta_{\text{max}} \\ 0, & \text{otherwise} \end{cases} \quad (10)$$

where only components that satisfy the contribution $\delta_i = \delta_{\text{max}}$ are allocated computational resources. $\mathcal{J}_{\min}$ is the minimum number of iterations for the component with the least difficulty, $\mathcal{J}_{\text{inc}}$ represents the maximum increment of the number of iterations, and both $\mathcal{J}_{\min}$ and $\mathcal{J}_{\text{inc}}$ are set to 100 in the experiment. $\alpha_i \in [0, 1]$ is the weight, which depends on the relative difficulty $(d_i - d_{\min})$ of the component. The greater the difficulty d_i of the component, the greater the weight α_i , and the more computational resources the component is allocated to. α_i is described as follows:

$$\alpha_i = (d_i - d_{\min})^\beta \quad (11)$$

where $d_{\min}$ is the minimum difficulty of the components in the set Φ , β is the exponent dependent on $d_{\min}$. The inverse sigmoid function is employed to establish the monotonic mapping $(0, 1) \mapsto [0, +\infty)$ from $d_{\min}$ to β

$$\beta = \max \left\{ 1 - \ln \left(\frac{d_{\min}}{1 - d_{\min}} \right), 0 \right\} \quad (12)$$

where the function $\max\{\cdot\}$ is to ensure that β is non-negative. The weight α_i is set to 0 if $d_{\min} = 0$, and α_i is set to 1 if $d_{\min} = 1$.

Fig. 2 displays the change curves of the weight α corresponding to different exponents β . The convex curve corresponding to $\beta > 1$ indicates the minimum difficulty $d_{\min} < 0.5$ of the components in the set Φ . In this case, simple components are allocated fewer additional computational resources, and the additional computational resources allocated to the difficult components are significantly more than simple components. For example, the weight α is 0.027 when the relative

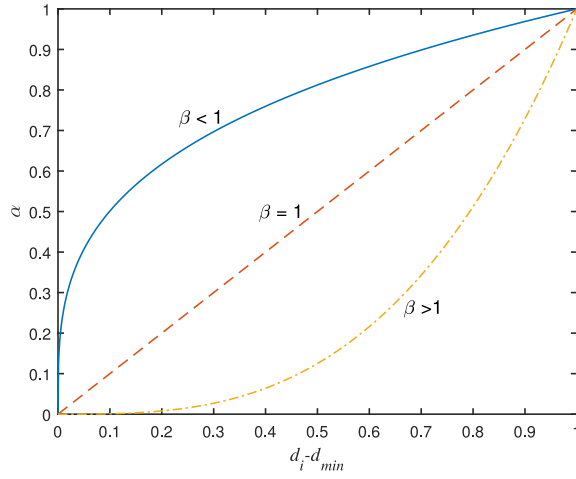


Fig. 2. Curves of weight α corresponding to different exponents β . $\beta = 0.3$, $\beta = 1$, and $\beta = 3$ are taken as the examples to illustrate the cases of $\beta < 1$, $\beta = 1$, and $\beta > 1$, respectively.

Algorithm 1 DCCC

```

1: FEs = 0, initialize parameters;
2:  $\{C_1, \dots, C_K\}$ , usedFEs = decomposition( $F$ );
3: FEs = FEs + usedFEs;
4: Initialize and evaluate population  $P$ ;
5: FEs = FEs + NP;
6: for  $i = 1$  to  $K$  do
7:   Optimize the component  $C_i$ ;
8:   FEs = FEs +  $\mathcal{J}_{\min} \cdot NP$ ;
9:   Get the difficulty  $d_i$  and contribution  $\delta_i$  of  $C_i$ ;
10: end for
11: while FEs < MaxFEs do
12:    $\delta_{\max} = \max\{\delta_1, \dots, \delta_K\}$ ;
13:    $\Phi = \{C_i | \delta_i = \delta_{\max}, i = 1, \dots, K\}$ ;
14:   Calculate the iterations  $\mathcal{J}_i$  of  $C_i$ ,  $C_i \in \Phi$ ;
15:   for  $C_i \in \Phi$  do
16:     Optimize the component  $C_i$ ;
17:     FEs = FEs +  $\mathcal{J}_i \cdot NP$ ;
18:     Update the difficulty  $d_i$  and contribution  $\delta_i$  of  $C_i$ ;
19:   end for
20: end while
21: return the best solution  $\mathbf{x}^*$  in the population  $P$ ;

```

difficulty is 0.3, and the weight α exceeds 0.5 when the relative difficulty is greater than 0.8.

Conversely, the concave curve corresponding to $\beta < 1$ represents the minimum difficulty $d_{\min} > 0.5$ of the components in the set Φ , which means the difficulty of all components in Φ is high. Consequently, even a small relative difficulty could cause a significant difference in computational resource allocation. For example, the weight α is close to 0.7 when the relative difficulty is 0.3.

The pseudocode of DCCC is shown in Algorithm 1, and the details of DCCC are described as follows.

First, the algorithm parameters are initialized and the number of used fitness evaluations FEs is set to 0. Then, DCCC employs a decomposition method to obtain a division $C = \{C_1, \dots, C_K\}$ of the problem F and FEs is updated, where

usedFEs is the number of fitness evaluations consumed by the decomposition algorithm. Next, DCCC randomly generates a set of candidate solutions with the size of NP in the search space as the initial population and evaluates it.

The optimization phase follows the start phase. To obtain the initial difficulty and contribution of each component, the components in C are traversed and optimized in turn, where the difficulty of the component is calculated as (8), and the contribution is calculated as (9). Here, the number of iterations of a cycle of each component is set to $\mathcal{J}_{\min}$.

In lines 11 to 20, DCCC selects one or more components according to the difficulty and contribution to optimize. First, all the components with the largest contribution $\delta_{\max}$ are selected to form the set Φ . Then, the number of iterations $\mathcal{J}$ of each component in the set Φ in the next cycle is calculated according to (10)–(12). Finally, DCCC optimizes the components in the set Φ in turn, and updates their difficulty and contribution. Note that the remaining available number of the fitness evaluation should be calculated before optimizing the component each time.

The algorithm is terminated when the number of available fitness evaluations is exhausted, and the optimal solution in the population is returned as the final result.

Fig. 3 compares the computational resource allocation of DCCC and three typical CC frameworks, namely, traditional CC, DBCC, and CBCC3. CC optimizes components in a round-robin manner [Fig. 3(a)]. DBCC only optimizes each component for one cycle, and allocates computational resources for each component according to difficulty in the start-up phase [Fig. 3(b)]. CBCC3 includes exploration and exploitation phases, and the component with the largest contribution is always prioritized for optimization in the exploitation phase [Fig. 3(c)]. For DCCC, as can be seen from Fig. 3(d), the first phase allocates equal amounts of computational resources to each component, and calculates the contribution δ and difficulty d for each component. In the second phase, DCCC selects multiple components with the largest $\delta_{\max}$ at the same time, and allocates unequal amounts of computational resources to each component for optimization based on their difficulty.

IV. EXPERIMENTS

In this section, our primary purpose is to evaluate the performance of the proposed DCCC. We test the performance of the DCCC framework on the well-known large-scale optimization benchmarks, i.e., CEC'2010 and CEC'2013 large-scale optimization benchmark suites [30], [40], and the results of DCCC are compared with several typical CC frameworks and state-of-the-art non-CC algorithms.

A. Benchmark

CEC'2010 and CEC'2013 large-scale optimization benchmark suites are widely employed to evaluate the performance of large-scale optimization algorithms, consisting of 20 and 15 LSOPs, respectively, [30], [40]. According to the separability of the problems, the benchmark suites could be divided

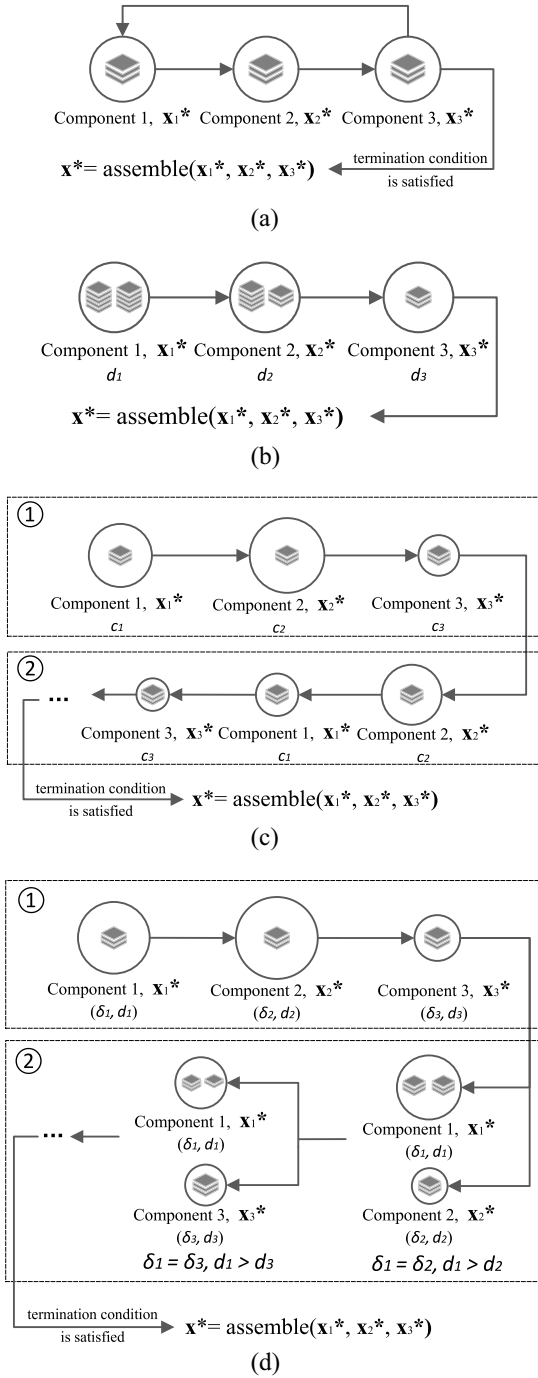


Fig. 3. Schematic of computational resource allocation schemes of CC, CBCC3, DBCC, and DCCC, where the chips inside of the circle represents the amount of computational resources obtained by the component, the size of the circle in CBCC3 and DCCC represents the fitness improvements to overall fitness value of the component in the current cycle, $\mathbf{x}_1^*$, $\mathbf{x}_2^*$, and $\mathbf{x}_3^*$ is the best solutions of the associated components, and $\mathbf{x}^*$ is best solution found. (a) Traditional CC. (b) DBCC. d_1 , d_2 , and d_3 are the difficulty of components 1, 2, and 3, respectively, and holds $d_1 > d_2 > d_3$. (c) CBCC3. c_1 , c_2 , and c_3 are the contribution of components 1, 2, and 3, respectively. (d) DCCC. (δ_1, d_1) , (δ_2, d_2) , and (δ_3, d_3) are the contribution and difficulty of components 1, 2, and 3, respectively.

into three categories, fully separable functions, partially separable functions, and nonseparable functions. In CEC'2013 benchmark suite, the nonseparable functions could be further

TABLE I
DESCRIPTION OF BENCHMARK FUNCTIONS

CEC'2010	CEC'2013	Feature	Dimension
F_1 - F_3	F_1 - F_3	Fully separable	1000
F_4 - F_{18}	F_4 - F_{11}	Partially separable	1000
-	F_{12} F_{13}, F_{14}	Overlap	1000 905
F_{19} - F_{20}	F_{15}	Fully nonseparable	1000

classified into overlapping functions and fully nonseparable functions. The classification of the benchmark functions is listed in Table I.

It should be emphasized that CEC'2010 benchmark suite has no imbalanced feature, and CEC'2013 benchmark suite has imbalances in the imbalance in the component size and the component weights. The former feature makes the difficulty of the components inconsistent, and the latter feature makes the influence of the components inconsistent. For the detailed descriptions of the benchmark suites, please refer to reports [30] and [40].

The maximum number of the fitness evaluations for each function is set to 3×10^6 suggested by [30] and [40], which is adopted as the termination condition for one run of the large-scale optimization algorithm. Moreover, each algorithm runs 25 times independently on each function, and the means and the standard deviations are counted to compare the performance.

B. Experimental Design

To demonstrate the performance of the proposed DCCC, five typical CC frameworks for large-scale optimization are employed as compared algorithms, including the traditional CC [11], CCFR [24], CBCC1, CBCC2 [22], and CBCC3 [23]. DBCC [28] does not participate in the comparison, because it was developed for many-modal optimization. To ensure fairness, both DCCC and the compared CC frameworks use SaNSDE [41] as the optimizer. We use ideal decomposition for all CC frameworks, and the separable variables of each function are divided into multiple 50-dimensional components. Moreover, the population size of all CC frameworks is set to 50 in this experiment. Note that this set of experiments is designed to compare the computational resource allocation strategies of different CC frameworks, so only separable problems are used as test functions, i.e., F_1 - F_{18} in CEC'2010 suite and F_1 - F_{11} in CEC'2013 suite.

Next, we use RDG2 [21] as the decomposition strategy and use SHADE [42] as the optimizer for DCCC, and the population size is set to 100. In this set of comparison experiments, three top algorithms from the CEC large-scale global optimization competitions, the winner of the CEC'2010, MA-SW-Chains [43], the winner of the CEC'2013 and CEC'2015, MOS-CEC2013 [44], and the runner-up of CEC'2018, MLSHADE-SPA [45], as well as two state-of-the-art algorithms, LLSO [46] and APSO-DEE [47] are employed as the compared algorithms.

TABLE II
MEANS AND STANDARD DEVIATIONS OF THE FITNESS VALUES OBTAINED BY DCCC AND COMPARED CC FRAMEWORKS WITH IDEAL DECOMPOSITION ON THE CEC'2010 AND CEC'2013 BENCHMARK SUITES OVER 25 INDEPENDENT RUNS

Function	DCCC	CC	CBCC1	CBCC2	CCFR	CBCC3
CEC'2010 benchmark suite						
F_1	3.24E-23 $\pm$ 2.75E-23	2.10E-23 $\pm$ 2.49E-23	9.90E+06 $\pm$ 1.30E+07	9.90E+06 $\pm$ 1.30E+07	1.20E-05 $\pm$ 4.90E-06	6.07E-23 $\pm$ 5.41E-23
F_2	8.28E+00 $\pm$ 7.28E+00	1.09E+01 $\pm$ 8.24E+00	4.70E+03 $\pm$ 4.80E+02	4.70E+03 $\pm$ 4.80E+02	2.70E+01 $\pm$ 5.20E+00	1.42E+01 $\pm$ 8.33E+00
F_3	1.06E-12 $\pm$ 4.03E-13	2.27E-13 $\pm$ 1.34E-14	1.20E+01 $\pm$ 3.70E-01	1.20E+01 $\pm$ 3.70E-01	4.60E+00 $\pm$ 4.60E-01	1.97E-11 $\pm$ 5.29E-11
F_4	6.96E+09 $\pm$ 4.36E+09	1.15E+12 $\pm$ 5.45E+11	6.00E+10 $\pm$ 4.40E+10	9.90E+10 $\pm$ 2.70E+10	8.30E+10 $\pm$ 6.20E+10	9.44E+09 $\pm$ 6.16E+09
F_5	7.04E+07 $\pm$ 1.08E+07	1.45E+08 $\pm$ 1.38E+07	6.80E+07 $\pm$ 7.00E+07	6.70E+07 $\pm$ 9.10E+06	7.20E+07 $\pm$ 1.30E+07	6.83E+07 $\pm$ 1.27E+07
F_6	1.37E-08 $\pm$ 2.42E-09	9.10E-09 $\pm$ 3.09E-09	1.30E+06 $\pm$ 6.40E+05	1.30E+06 $\pm$ 6.80E+05	7.70E+05 $\pm$ 7.10E+05	2.80E-08 $\pm$ 1.01E-08
F_7	7.25E-20 $\pm$ 3.27E-20	4.21E+05 $\pm$ 3.38E+05	5.90E+04 $\pm$ 9.30E+03	8.40E+04 $\pm$ 1.90E+04	1.50E-03 $\pm$ 2.50E-04	5.73E-20 $\pm$ 2.00E-20
F_8	7.97E+05 $\pm$ 1.63E+06	5.03E+07 $\pm$ 2.80E+07	8.60E+05 $\pm$ 1.60E+06	1.00E+06 $\pm$ 1.70E+06	3.20E+05 $\pm$ 1.10E+06	6.38E+05 $\pm$ 1.49E+06
F_9	3.56E+06 $\pm$ 5.21E+05	9.35E+06 $\pm$ 1.21E+06	1.70E+07 $\pm$ 2.10E+07	2.80E+09 $\pm$ 1.80E+09	9.40E+06 $\pm$ 1.20E+06	3.62E+06 $\pm$ 3.71E+05
F_{10}	1.39E+03 $\pm$ 1.19E+02	1.46E+03 $\pm$ 6.29E+01	3.00E+03 $\pm$ 1.70E+02	4.50E+03 $\pm$ 6.60E+02	1.40E+03 $\pm$ 1.00E+02	1.44E+03 $\pm$ 1.75E+02
F_{11}	5.60E-13 $\pm$ 2.66E-13	2.14E-13 $\pm$ 1.24E-14	2.20E+01 $\pm$ 3.20E+00	2.40E+01 $\pm$ 2.70E+00	1.00E+01 $\pm$ 2.70E+00	3.52E-02 $\pm$ 1.76E-01
F_{12}	1.01E-03 $\pm$ 8.27E-04	5.63E+00 $\pm$ 1.48E+00	1.80E+04 $\pm$ 6.50E+03	2.50E+04 $\pm$ 7.30E+03	1.20E+00 $\pm$ 4.60E+00	7.46E-04 $\pm$ 2.85E-04
F_{13}	1.81E+02 $\pm$ 6.78E+01	5.01E+02 $\pm$ 1.03E+02	1.90E+04 $\pm$ 6.30E+03	2.80E+04 $\pm$ 5.40E+03	3.20E+02 $\pm$ 9.90E+01	1.46E+02 $\pm$ 8.34E+01
F_{14}	1.91E+07 $\pm$ 1.34E+06	2.01E+07 $\pm$ 1.95E+06	2.80E+07 $\pm$ 2.10E+06	9.50E+09 $\pm$ 5.20E+08	2.50E+07 $\pm$ 2.90E+06	1.89E+07 $\pm$ 1.18E+06
F_{15}	3.07E+03 $\pm$ 1.98E+02	2.93E+03 $\pm$ 1.18E+02	4.00E+03 $\pm$ 1.50E+02	4.20E+03 $\pm$ 1.60E+02	2.80E+03 $\pm$ 1.30E+02	3.34E+03 $\pm$ 2.46E+02
F_{16}	4.11E-02 $\pm$ 2.05E-01	2.02E-13 $\pm$ 1.42E-14	1.90E+01 $\pm$ 3.20E+00	2.00E+01 $\pm$ 3.40E+00	2.00E+01 $\pm$ 2.60E+00	2.37E-13 $\pm$ 1.32E-14
F_{17}	7.74E+00 $\pm$ 2.08E+00	1.14E+01 $\pm$ 3.44E+00	3.50E+01 $\pm$ 4.90E+01	1.40E+02 $\pm$ 4.40E+01	9.80E+00 $\pm$ 1.10E+01	7.98E+00 $\pm$ 1.64E+00
F_{18}	9.66E+02 $\pm$ 1.10E+02	9.65E+02 $\pm$ 1.12E+02	1.10E+03 $\pm$ 1.80E+02	1.40E+03 $\pm$ 1.90E+02	1.10E+03 $\pm$ 1.80E+02	9.31E+02 $\pm$ 1.36E+02
+ / $\approx$ / -		11/0/7	17/0/1	17/0/1	16/0/2	10/0/8
CEC'2013 benchmark suite						
F_1	3.99E-23 $\pm$ 3.22E-23	3.90E-23 $\pm$ 4.28E-23	1.40E+07 $\pm$ 3.60E+07	1.40E+07 $\pm$ 3.60E+07	1.30E-05 $\pm$ 3.20E-06	3.61E-23 $\pm$ 2.86E-23
F_2	1.87E+01 $\pm$ 8.98E+00	3.00E+01 $\pm$ 8.93E+00	2.10E+04 $\pm$ 9.90E+02	2.10E+04 $\pm$ 9.90E+02	5.50E-01 $\pm$ 1.50E+00	2.63E+01 $\pm$ 9.93E+00
F_3	2.00E+01 $\pm$ 1.65E-03	2.00E+01 $\pm$ 8.62E-04	2.10E+01 $\pm$ 1.10E-02	2.10E+01 $\pm$ 1.10E-02	2.00E+01 $\pm$ 3.10E-07	2.00E+01 $\pm$ 5.52E-03
F_4	1.30E+07 $\pm$ 4.61E+06	5.93E+09 $\pm$ 2.36E+10	1.60E+08 $\pm$ 6.00E+07	6.60E+10 $\pm$ 5.60E+09	4.50E+07 $\pm$ 1.70E+07	1.25E+07 $\pm$ 4.63E+06
F_5	2.11E+06 $\pm$ 3.95E+05	4.02E+06 $\pm$ 5.33E+05	2.50E+06 $\pm$ 4.20E+05	2.40E+06 $\pm$ 4.50E+05	2.50E+06 $\pm$ 2.70E+05	2.13E+06 $\pm$ 4.49E+05
F_6	1.05E+06 $\pm$ 1.10E+04	1.06E+06 $\pm$ 1.13E+03	1.10E+06 $\pm$ 1.90E+03	1.10E+06 $\pm$ 1.70E+03	1.10E+06 $\pm$ 1.20E+03	1.05E+06 $\pm$ 5.74E+03
F_7	1.49E+07 $\pm$ 2.21E+07	1.58E+08 $\pm$ 1.29E+08	1.90E+07 $\pm$ 2.40E+07	9.60E+07 $\pm$ 3.70E+08	8.60E+06 $\pm$ 1.90E+07	1.80E+07 $\pm$ 2.80E+07
F_8	5.55E+09 $\pm$ 4.73E+09	7.45E+13 $\pm$ 4.26E+13	2.00E+13 $\pm$ 2.80E+13	1.00E+12 $\pm$ 1.30E+11	9.60E+09 $\pm$ 1.60E+10	1.28E+10 $\pm$ 2.61E+10
F_9	1.59E+08 $\pm$ 2.96E+07	3.04E+08 $\pm$ 6.02E+07	2.50E+08 $\pm$ 3.80E+07	2.20E+08 $\pm$ 2.80E+07	1.90E+08 $\pm$ 2.80E+07	1.64E+08 $\pm$ 2.96E+07
F_{10}	9.24E+07 $\pm$ 1.02E+06	9.43E+07 $\pm$ 3.07E+05	9.40E+07 $\pm$ 2.80E+05	9.40E+07 $\pm$ 2.30E+05	9.50E+07 $\pm$ 1.90E+05	9.27E+07 $\pm$ 7.43E+05
F_{11}	5.65E+08 $\pm$ 2.82E+08	1.11E+10 $\pm$ 1.77E+10	3.00E+09 $\pm$ 1.00E+10	4.90E+10 $\pm$ 9.50E+10	3.30E+08 $\pm$ 3.20E+08	6.31E+08 $\pm$ 2.67E+08
+ / $\approx$ / -		9/1/1	11/0/0	11/0/0	7/1/3	7/2/2

The offspring of the subpopulation generated in the previous cycle is used to estimate the difficulty of the components.

C. Experimental Results

Tables II and III list the statistical results obtained by DCCC and several compared algorithms on CEC'2010 and CEC'2013 benchmark suites, where the best result is highlighted in bold for each function. Moreover, Tables II and III also count the number of wins in DCCC against other compared algorithms on two sets of benchmark functions, where the symbols “+ / $\approx$ / -” in the last row of tables represent that DCCC is better than/no different/worse than the compared algorithms.

1) *Comparisons of DCCC and Other CC Frameworks:* Table II shows the statistical results obtained by DCCC and several CC frameworks on the separable functions, in which all CC frameworks use ideal decomposition.

From the comparison on CEC'2010 benchmark suite, the results demonstrate that DCCC is significantly better than the

algorithms CBCC1, CBCC2, and CCFR, winning 17, 17, and 15 times out of 18 separable functions. Moreover, DCCC is also superior to the traditional CC framework and CBCC3, performing better on 11 and 10 functions, more than half of the test functions.

The components of the functions in CEC'2013 benchmark suite are imbalanced. The results demonstrate that DCCC is rather competitive against other compared CC frameworks on this benchmark suite. DCCC performs better than CBCC1 and CBCC2 on all 11 separable functions, and outperforms the traditional CC framework on nine functions. Compared with CCFR and CBCC3, the performance of DCCC is also satisfactory, achieving results of “7/1/3” and “7/2/2”, respectively. The performance of DCCC is worse than CCFR on F_2 , F_7 , and F_{11} , and worse than CBCC3 F_1 and F_4 . Experimental results support that DCCC is better at dealing with problems with imbalanced components, including the imbalance in component influence and difficulty. Note that the imbalance

TABLE III
MEANS AND STANDARD DEVIATIONS OF THE FITNESS VALUES OBTAINED BY DCCC AND COMPARED STATE-OF-THE-ART LARGE-SCALE OPTIMIZATION ALGORITHMS ON THE CEC'2010 AND CEC'2013 BENCHMARK SUITES OVER 25 INDEPENDENT RUNS

Function	DCCC	MA-SW-Chains	MOS-CEC2013	LSHADE-SPA	LLSO	APSO-DEE
CEC'2010 benchmark suite						
F_1	5.47E-24 $\pm$ 6.98E-24	3.88E-14 $\pm$ 3.59E-14	0.00E+00 $\pm$ 0.00E+00	4.01E-23 $\pm$ 2.88E-23	3.13E-22 $\pm$ 8.03E-23	1.06E-20 $\pm$ 2.55E-22
F_2	1.00E+00 $\pm$ 1.26E+00	8.63E+02 $\pm$ 5.84E+01	0.00E+00 $\pm$ 0.00E+00	4.70E+01 $\pm$ 7.83E+00	9.82E+02 $\pm$ 4.39E+01	5.86E+02 $\pm$ 4.43E+00
F_3	1.75E+01 $\pm$ 2.44E-01	5.41E-13 $\pm$ 2.13E-13	1.65E-12 $\pm$ 6.73E-14	9.84E-14 $\pm$ 9.70E-15	2.76E-14 $\pm$ 2.38E-15	3.83E-13 $\pm$ 6.86E-15
F_4	1.38E+07 $\pm$ 1.57E+07	2.94E+11 $\pm$ 9.32E+10	1.56E+10 $\pm$ 6.02E+09	1.84E+11 $\pm$ 6.32E+10	4.40E+11 $\pm$ 1.10E+11	2.75E+11 $\pm$ 7.76E+09
F_5	4.17E+07 $\pm$ 7.30E+06	1.75E+08 $\pm$ 1.03E+08	1.11E+08 $\pm$ 2.25E+07	6.07E+07 $\pm$ 9.52E+06	1.22E+07 $\pm$ 3.43E+06	9.91E+06 $\pm$ 3.96E+05
F_6	1.92E+06 $\pm$ 4.39E+05	3.52E+04 $\pm$ 1.72E+05	1.22E-07 $\pm$ 6.43E-08	6.60E-09 $\pm$ 1.29E-09	5.20E-01 $\pm$ 7.46E-01	3.73E-09 $\pm$ 9.06E-12
F_7	2.13E-21 $\pm$ 9.99E-22	3.30E+02 $\pm$ 1.40E+03	0.00E+00 $\pm$ 0.00E+00	8.08E-06 $\pm$ 7.44E-06	7.19E+02 $\pm$ 2.59E+03	6.18E-02 $\pm$ 7.92E-03
F_8	1.12E+06 $\pm$ 1.83E+06	9.28E+06 $\pm$ 2.36E+07	1.95E+00 $\pm$ 8.03E+00	4.43E+03 $\pm$ 2.30E+04	2.34E+07 $\pm$ 2.46E+05	7.01E+06 $\pm$ 3.18E+05
F_9	7.88E+05 $\pm$ 1.33E+05	1.15E+07 $\pm$ 1.59E+06	3.46E+06 $\pm$ 4.49E+05	1.65E+07 $\pm$ 2.34E+06	4.36E+07 $\pm$ 4.28E+06	1.75E+07 $\pm$ 2.07E+05
F_{10}	5.62E+02 $\pm$ 2.84E+01	2.06E+03 $\pm$ 1.19E+02	3.78E+03 $\pm$ 1.47E+02	5.01E+03 $\pm$ 2.72E+02	8.91E+02 $\pm$ 3.66E+01	5.98E+02 $\pm$ 4.23E+00
F_{11}	3.47E+01 $\pm$ 1.91E+00	3.69E+01 $\pm$ 8.24E+00	1.91E+02 $\pm$ 4.07E-01	8.96E+01 $\pm$ 1.38E+01	5.80E+00 $\pm$ 5.40E+00	8.27E-13 $\pm$ 2.04E-14
F_{12}	6.96E-12 $\pm$ 4.81E-12	3.19E-06 $\pm$ 5.32E-07	0.00E+00 $\pm$ 0.00E+00	4.35E+01 $\pm$ 9.61E+00	1.25E+04 $\pm$ 1.46E+03	1.62E+04 $\pm$ 3.86E+02
F_{13}	1.07E+01 $\pm$ 5.11E+00	1.09E+03 $\pm$ 6.29E+02	7.14E+02 $\pm$ 5.68E+02	9.95E+01 $\pm$ 8.47E+01	7.35E+02 $\pm$ 1.93E+02	5.69E+02 $\pm$ 2.47E+01
F_{14}	4.66E+06 $\pm$ 5.47E+05	3.34E+07 $\pm$ 3.37E+06	9.80E+06 $\pm$ 6.03E+05	6.02E+07 $\pm$ 6.03E+06	1.24E+08 $\pm$ 7.38E+06	5.95E+07 $\pm$ 5.51E+05
F_{15}	1.14E+03 $\pm$ 4.89E+01	2.69E+03 $\pm$ 9.75E+01	7.44E+03 $\pm$ 1.84E+02	3.77E+03 $\pm$ 9.59E+01	8.33E+02 $\pm$ 4.31E+01	6.41E+02 $\pm$ 4.08E+00
F_{16}	3.76E+01 $\pm$ 2.22E+00	1.08E+02 $\pm$ 1.51E+01	3.82E+02 $\pm$ 1.55E+01	2.49E+02 $\pm$ 1.65E+01	4.25E+00 $\pm$ 2.41E+00	7.16E-13 $\pm$ 2.23E-14
F_{17}	3.16E-05 $\pm$ 1.21E-05	1.26E+00 $\pm$ 9.45E-02	2.83E-07 $\pm$ 7.97E-08	6.16E+02 $\pm$ 9.20E+01	9.05E+04 $\pm$ 3.53E+03	9.37E+04 $\pm$ 1.34E+03
F_{18}	2.90E+02 $\pm$ 6.67E+01	1.87E+03 $\pm$ 5.79E+02	1.54E+03 $\pm$ 7.46E+02	5.73E+02 $\pm$ 1.76E+02	2.55E+03 $\pm$ 8.32E+02	1.21E+03 $\pm$ 4.43E+01
F_{19}	4.61E+05 $\pm$ 4.29E+04	2.85E+05 $\pm$ 1.74E+04	2.91E+04 $\pm$ 2.14E+03	4.51E+05 $\pm$ 2.34E+04	1.80E+06 $\pm$ 9.96E+04	1.42E+06 $\pm$ 1.37E+04
F_{20}	1.01E+08 $\pm$ 1.10E+08	5.71E+06 $\pm$ 7.59E+01	3.52E+02 $\pm$ 4.43E+02	1.26E+02 $\pm$ 9.55E+01	1.88E+03 $\pm$ 1.90E+02	1.02E+03 $\pm$ 9.89E+00
+ / $\approx$ / -		16/0/4	10/0/10	15/0/5	13/0/7	13/0/7
CEC'2013 benchmark suite						
F_1	7.50E-24 $\pm$ 1.06E-23	8.49E-13 $\pm$ 1.09E-12	1.27E-22 $\pm$ 7.41E-23	1.94E-22 $\pm$ 4.79E-22	3.99E-22 $\pm$ 1.32E-22	4.14E-20 $\pm$ 3.62E-21
F_2	3.70E+00 $\pm$ 2.74E+00	1.22E+03 $\pm$ 1.14E+02	8.32E+02 $\pm$ 4.48E+01	7.89E+01 $\pm$ 9.69E+00	1.14E+03 $\pm$ 5.78E+01	6.31E+02 $\pm$ 4.49E+00
F_3	2.00E+01 $\pm$ 6.82E-04	2.14E+01 $\pm$ 5.62E-02	9.18E-13 $\pm$ 5.12E-14	9.96E-14 $\pm$ 7.91E-15	2.16E+01 $\pm$ 4.07E-03	2.16E+01 $\pm$ 8.95E-04
F_4	8.44E+05 $\pm$ 5.74E+05	4.58E+09 $\pm$ 2.46E+09	1.74E+08 $\pm$ 7.87E+07	6.90E+08 $\pm$ 4.41E+08	6.68E+09 $\pm$ 1.68E+09	2.05E+09 $\pm$ 5.73E+07
F_5	1.42E+06 $\pm$ 3.64E+05	1.87E+06 $\pm$ 3.06E+05	6.94E+06 $\pm$ 8.85E+05	1.80E+06 $\pm$ 2.34E+05	7.00E+05 $\pm$ 1.28E+05	6.50E+05 $\pm$ 2.18E+04
F_6	1.05E+06 $\pm$ 1.49E+04	1.01E+06 $\pm$ 1.53E+04	1.48E+05 $\pm$ 6.43E+04	1.40E+03 $\pm$ 2.39E+03	1.06E+06 $\pm$ 8.28E+02	1.06E+06 $\pm$ 1.82E+02
F_7	2.03E-02 $\pm$ 8.86E-02	3.45E+06 $\pm$ 1.27E+06	1.62E+04 $\pm$ 9.10E+03	5.31E+04 $\pm$ 1.96E+04	1.60E+06 $\pm$ 8.38E+05	5.49E+05 $\pm$ 4.59E+04
F_8	2.57E+08 $\pm$ 9.80E+07	4.85E+13 $\pm$ 1.02E+13	8.00E+12 $\pm$ 3.07E+12	9.77E+12 $\pm$ 5.53E+12	1.20E+14 $\pm$ 3.35E+13	4.40E+13 $\pm$ 2.27E+12
F_9	1.04E+08 $\pm$ 2.10E+07	1.07E+08 $\pm$ 1.68E+07	3.83E+08 $\pm$ 6.29E+07	1.61E+08 $\pm$ 1.94E+07	1.30E+08 $\pm$ 3.97E+07	3.80E+07 $\pm$ 1.23E+06
F_{10}	9.22E+07 $\pm$ 1.18E+06	9.18E+07 $\pm$ 1.06E+06	9.02E+05 $\pm$ 5.07E+05	6.56E+02 $\pm$ 2.40E+02	9.40E+07 $\pm$ 2.11E+05	9.40E+07 $\pm$ 4.25E+04
F_{11}	2.49E+05 $\pm$ 7.57E+05	2.19E+08 $\pm$ 2.98E+07	5.22E+07 $\pm$ 2.05E+07	4.04E+07 $\pm$ 1.98E+07	9.30E+11 $\pm$ 9.50E+09	6.85E+07 $\pm$ 2.80E+06
F_{12}	1.15E+08 $\pm$ 1.69E+08	1.25E+03 $\pm$ 1.05E+02	2.47E+02 $\pm$ 2.54E+02	1.04E+02 $\pm$ 7.64E+01	1.79E+03 $\pm$ 1.90E+02	1.12E+03 $\pm$ 1.27E+01
F_{13}	3.55E+07 $\pm$ 3.81E+07	1.98E+07 $\pm$ 1.82E+06	3.40E+06 $\pm$ 1.06E+06	7.21E+07 $\pm$ 4.99E+07	3.35E+08 $\pm$ 1.71E+08	6.74E+07 $\pm$ 6.08E+06
F_{14}	3.11E+08 $\pm$ 5.23E+08	1.36E+08 $\pm$ 2.11E+07	2.56E+07 $\pm$ 7.94E+06	1.52E+07 $\pm$ 3.08E+06	1.72E+08 $\pm$ 1.38E+08	4.94E+07 $\pm$ 3.77E+06
F_{15}	2.19E+06 $\pm$ 3.18E+05	5.71E+06 $\pm$ 7.57E+05	2.35E+06 $\pm$ 1.94E+05	2.76E+07 $\pm$ 9.01E+06	4.48E+06 $\pm$ 3.32E+05	3.07E+06 $\pm$ 5.37E+04
+ / $\approx$ / -		10/0/5	9/0/6	10/0/5	12/0/3	11/0/4

in the component difficulty on CEC'2013 benchmark suite is manifested in the nonuniform size of the components.

2) *Comparisons of DCCC and Large-Scale Optimization Algorithms:* To comprehensively evaluate the performance of DCCC, Table III presents the comparison results among DCCC combined with RDG2 and SHADE and five state-of-the-art large-scale optimization algorithms. For CEC'2010 benchmark suite, DCCC achieves better results than MA-SW-Chains, LSHADE-SPA, LLSO, and APSO-DEE, beating them 16, 15, 13, and 13 times in a total of 20 functions, respectively. Moreover, DCCC outperforms MOS-CEC2013 on ten functions. For the CEC'2013 benchmark suite, the experimental results are superior to all compared algorithms,

winning 10, 9, 10, 12, and 11 times on 15 functions, respectively. Particularly, it can be seen that DCCC is significantly superior to the other algorithms on the separable functions (F_1 – F_{11}), the comparison results “+/-” between DCCC and the compared algorithms are “9/2,” “8/3,” “8/3,” “10/1,” and “9/2.”

There are several reasons for the poor performance of DCCC on some separable functions. First, RDG2 only works on the decomposition of the additive separable problems, whereas not nonadditive separable problems, such as F_6 and F_{11} in CEC'2010 benchmark suite as well as F_3 and F_6 in CEC'2013 benchmark suite. The inaccurate grouping scheme could degrade the performance of DCCC. Second, limited by

the convergence of the optimizer SHADE, DCCC is less competitive than compared algorithms in terms of the accuracy of the result, such as F_1 , F_7 , and F_{12} in CEC'2010 benchmark.

Overall, the experimental results support that DCCC combined with RDG2 and SHADE has significant advantages versus state-of-the-art large-scale optimization algorithms on CEC'2010 and CEC'2013 benchmark suites, especially in achieving satisfactory and competitive results on separable problems.

V. DISCUSSION

In this section, first, we show the performance of DCCC on heterogeneous LSOPs. Then, we discuss the computational resource allocation of DCCC. Finally, we carry out an ablation experiment to investigate the factors in the difficulty calculation (8). All experiments in this section adopt the same experimental setting as the first set of experiments in Section IV, that is, the ideal decomposition and SaNSDE are used, and the same parameters are applied.

A. Experiments on Heterogeneous LSOPs

The homogeneous design is used in CEC'2010 and CEC'2013 benchmark suites, that is, an LSOP consists of the same class of basis functions. However, there are a large number of LSOPs with heterogeneous structures in real-world applications [1], i.e., the different disciplinary contexts of the components result in significant differences in difficulty and explicit expressions of the components. Therefore, the DCCC is further tested on heterogeneous LSOPs in this section to demonstrate its performance.

We design two heterogeneous LSOPs by reference to the construction method of the partially additive separable functions II in the CEC'2013 large-scale optimization benchmark. The components $h_i, \dots, h_k$ of each heterogeneous LSOPs are selected from the unimodal functions (f_1 – f_5) and basic multimodal functions (f_6 – f_{18}) in the CEC'2013 global optimization benchmark [48], and the value range of the component is tuned. The form is as follows:

$$H(\mathbf{x}) = \sum_{i=1}^k w_i h_i(\mathbf{z}_i) \quad (13)$$

where w_i is the weight of i th components, k is the number of components, and the dimension of i th component is S_i . The vector $\mathbf{z}_i$ is a transformation of the vector $\mathbf{x}_i$, and $\{\mathbf{x}_1, \dots, \mathbf{x}_k\}$ is a partition of the shuffled $\mathbf{x}$. For designed heterogeneous LSOPs, the component dimension and the index of the components in the CEC'2013 global optimization benchmark are listed in Table IV.

The DCCC and several CC frameworks are tested on two benchmark problems. Fig. 4 displays the mean and standard deviation obtained by each CC framework in a bar graph and error bars, respectively, which visualizes the superiority of DCCC on two test heterogeneous LSOPs. For the test problem H_1 , DCCC is the winner in this set of comparisons. Particularly, the performance of DCCC is more stable compared to CBCC3. Moreover, Fig. 4(a) shows that the

TABLE IV
DETAILS OF HETEROGENEOUS LSOPs

Problems	Component size	Function index
H_1	{50, 25, 100, 50, 50, 100, 50, 50, 50, 50, 50, 50, 25, 50, 25, 50, 50, 25, 50,}	{ $f_9, f_{15}, f_{14}, f_8, f_4, f_2, f_{12}, f_6, f_8, f_{11}, f_9, f_{14}, f_{15}, f_{18}, f_6, f_{17}, f_8, f_{12}, f_{20}, f_2$ }
H_2	{50, 100, 25, 25, 50, 100, 25, 25, 50, 25, 100, 50, 50, 50, 50, 50, 25, 50,}	{ $f_8, f_2, f_{17}, f_{18}, f_8, f_{14}, f_{15}, f_{17}, f_8, f_{20}, f_{11}, f_{12}, f_9, f_{12}, f_6, f_9, f_{18}, f_{15}, f_{18}, f_2$ }

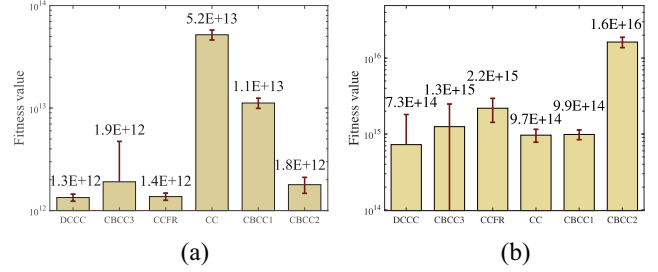


Fig. 4. Means and standard deviations of the fitness values obtained by DCCC and compared CC frameworks with ideal decomposition on the heterogeneous LSOPs over 25 independent runs. (a) H_1 . (b) H_2 .

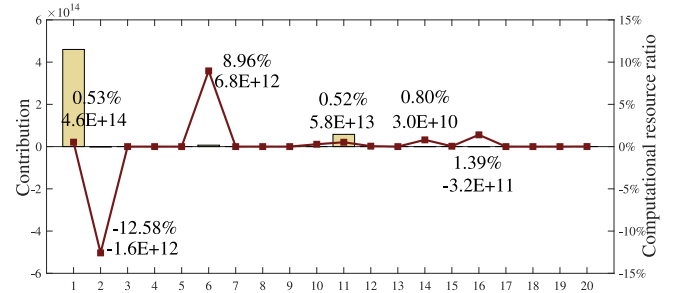


Fig. 5. Difference in contribution and computational resource ratio between DCCC and CBCC3 on 20 components of H_2 , where the left ordinate corresponds to the bar graph and the right ordinate corresponds to the line graph. Computational resource ratio differences less than 0.5% and contribution differences less than 1×10^{10} are not shown.

contribution-aware CC frameworks achieve better results than the round-robin CC framework. Fig. 4(b) displays that the performance of contribution-aware CC is worse instead on H_2 , which indicates that the contribution-aware CC neglects some influential components. Conversely, DCCC achieves competitive results on H_2 . Despite the large standard deviation, the advantage on the mean value indicates that DCCC could allocate computational resources for the truly influential components in most cases.

Fig. 5 further compares the investments and returns of DCCC and CBCC3 on each component of H_2 , corresponding to the line and bar graphs, respectively. It can be seen that DCCC invests more computational resources on components 1, 6, 11, 14, as well as 16 and obtains satisfactory returns on components 1, 6, and 11, especially on component 1, whereas CBCC3 allocates a large number of computational resources on component 2 without significant returns. Note that components 1 (f_8), 6 (f_{14}), 11 (f_{11}), 14 (f_{12}), and 16 (f_9) are multimodal functions, while component 2 (f_2) is a unimodal function. Generally, the former is more difficult than the latter.

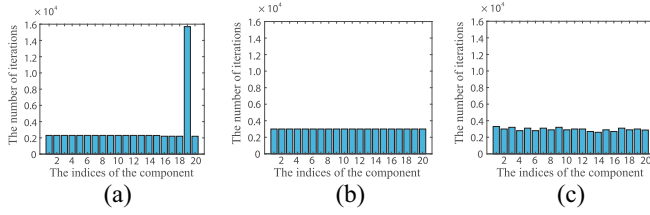


Fig. 6. Computational resource allocation of CBCC2, CC, and CBCC3 on CEC'2010 benchmark function F_{17} at a single run. (a) CBCC2. (b) Traditional CC. (c) CBCC3.

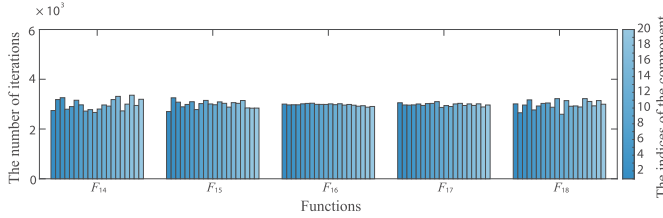


Fig. 7. Average of the computational resource allocation of DCCC on CEC'2010 benchmark partially separable functions F_{14} – F_{18} with 20 components over 25 independent runs.

Therefore, the experimental results demonstrate that difficult components are easily neglected by CBCC3 because of their small early contributions, and the computational resource allocation based on difficulty and contribution adopted by DCCC could avoid the above phenomenon.

B. Computational Resource Allocation

Fig. 6 displays the computational resource allocation of CBCC2, traditional CC, and CBCC3 on function F_{17} , and Fig. 7 displays the computational resource allocation of DCCC on five partially separable functions in CEC'2010 suite. Considering that the components of CEC'2010 benchmark functions are balanced, excessive preference for optimizing a few components leads to poor results, such as CBCC2. On the contrary, the traditional CC and CBCC3 achieve better results, because the former allocates the same computational resources to each component, and the allocation of the latter is approximately equal. Fig. 7 demonstrates that the computational resource allocation of DCCC is almost uniform for all components, especially functions F_{16} and F_{17} . In other words, DCCC could handle problems with balanced components without over preference.

Fig. 8 shows the dynamic allocation of the computational resource of DCCC on partially separable functions F_8 – F_{11} in the CEC'2013 suite, where the size of the circle indicates the number of fitness evaluations allocated to components at one cycle. Fig. 9 displays the overall computational resource allocation of DCCC on these functions in the form of a box plot, and the weight of each component is marked with a line plot. Note that the CEC'2013 benchmark weights the 20 components of each function to distinguish their influence on the overall fitness value. The larger the weight, the greater the influence. It can be seen that DCCC always gives priority to components with significant contributions, such as function F_{10} , and concerns the difficulty of the components

TABLE V
ABLATION STUDY ON THE DIFFICULTY CALCULATION OF THE COMPONENTS. THE MEANS AND STANDARD DEVIATIONS OF THE FITNESS VALUES OBTAINED BY DCCC, DCCC OMITTED THE FACTOR p_s , AND DCCC OMITTED THE FACTOR r ON 13 LSOPs OVER 25 INDEPENDENT RUNS

Function	DCCC	DCCC (omitted p_s)	DCCC (omitted r)
F_1	$3.99\text{E-}23 \pm 3.22\text{E-}23$	$3.53\text{E-}23 \pm 3.74\text{E-}23$	$4.52\text{E-}23 \pm 3.41\text{E-}23$
F_2	$1.87\text{E+}01 \pm 8.98\text{E+}00$	$2.87\text{E+}01 \pm 9.72\text{E+}00$	$1.97\text{E+}01 \pm 9.31\text{E+}00$
F_3	$2.00\text{E+}01 \pm 1.65\text{E-}03$	$2.00\text{E+}01 \pm 3.88\text{E-}03$	$2.00\text{E+}01 \pm 2.33\text{E-}03$
F_4	$1.30\text{E+}07 \pm 4.61\text{E+}06$	$1.29\text{E+}07 \pm 6.35\text{E+}06$	$1.31\text{E+}07 \pm 5.16\text{E+}06$
F_5	$2.11\text{E+}06 \pm 3.95\text{E+}05$	$2.15\text{E+}06 \pm 3.39\text{E+}05$	$2.13\text{E+}06 \pm 3.74\text{E+}05$
F_6	$1.05\text{E+}06 \pm 1.10\text{E+}04$	$1.05\text{E+}06 \pm 1.10\text{E+}04$	$1.05\text{E+}06 \pm 1.53\text{E+}03$
F_7	$1.49\text{E+}07 \pm 2.21\text{E+}07$	$2.01\text{E+}07 \pm 2.46\text{E+}07$	$1.70\text{E+}07 \pm 2.37\text{E+}07$
F_8	$5.55\text{E+}09 \pm 4.73\text{E+}09$	$9.97\text{E+}09 \pm 2.25\text{E+}10$	$3.43\text{E+}10 \pm 8.56\text{E+}10$
F_9	$1.59\text{E+}08 \pm 2.96\text{E+}07$	$1.51\text{E+}08 \pm 2.27\text{E+}07$	$1.62\text{E+}08 \pm 2.64\text{E+}07$
F_{10}	$9.24\text{E+}07 \pm 1.02\text{E+}06$	$9.24\text{E+}07 \pm 9.72\text{E+}05$	$9.24\text{E+}07 \pm 9.64\text{E+}05$
F_{11}	$5.65\text{E+}08 \pm 2.82\text{E+}08$	$5.63\text{E+}08 \pm 2.56\text{E+}08$	$5.03\text{E+}08 \pm 3.21\text{E+}08$
H_1	$1.34\text{E+}12 \pm 1.06\text{E+}11$	$1.36\text{E+}12 \pm 1.34\text{E+}11$	$1.32\text{E+}12 \pm 1.06\text{E+}11$
H_2	$7.25\text{E+}14 \pm 1.08\text{E+}15$	$8.32\text{E+}14 \pm 1.03\text{E+}15$	$7.82\text{E+}14 \pm 1.03\text{E+}15$
+/ ≈ /–		6/3/4	8/3/2

when the contribution is of the same magnitude. The size of the circle demonstrates the decision behavior of DCCC for components with different difficulties. DCCC allocated the maximum number of fitness evaluations to the components in most cycles when all subproblems were overly difficult, such as F_9 , while there are differences in computational resource allocation to the components of functions F_8 and F_{11} because the base functions of them are relatively easy. Fig. 9 demonstrates that for the four partially separable functions, the computational resource allocation of DCCC to each component maintains the same trend as its weight. This demonstrates that DCCC satisfies the principle that more computational resources are allocated to components with greater weight. Moreover, DCCC considers the difficulty of components. For example, for two components 12 and 18 with similar weights in function F_9 , DCCC allocates more computational resources to the 100D component 18 than the 25D component 12, because component 18 is more difficult. The similar situations also occur on other functions, such as component 5 (100D) and component 16 (25D) of function F_8 , and the component 17 (25D) and component 19 (50D) of function F_{11} . The number of computational resources allocated to component 5 is more than that of component 16 on function F_8 , and the number of computational resources allocated to component 19 is more than that of component 17 on function F_{11} .

C. Ablation Study

DCCC adopts two factors to estimate the subproblem difficulty, i.e., the difficulty of the fitness landscape and the behaviors of the optimization algorithm, as measured by the FDC coefficient r and the success probability p_s , respectively. In this section, we carry out the ablation study to analyze the significance of the factors r and p_s . The control experiments omit p_s as well as r in the difficulty calculation (8), respectively.

Table V presents the experimental results of DCCC and two ablation experiments on 13 large-scale optimization benchmark problems, including 11 partially separable homogeneous

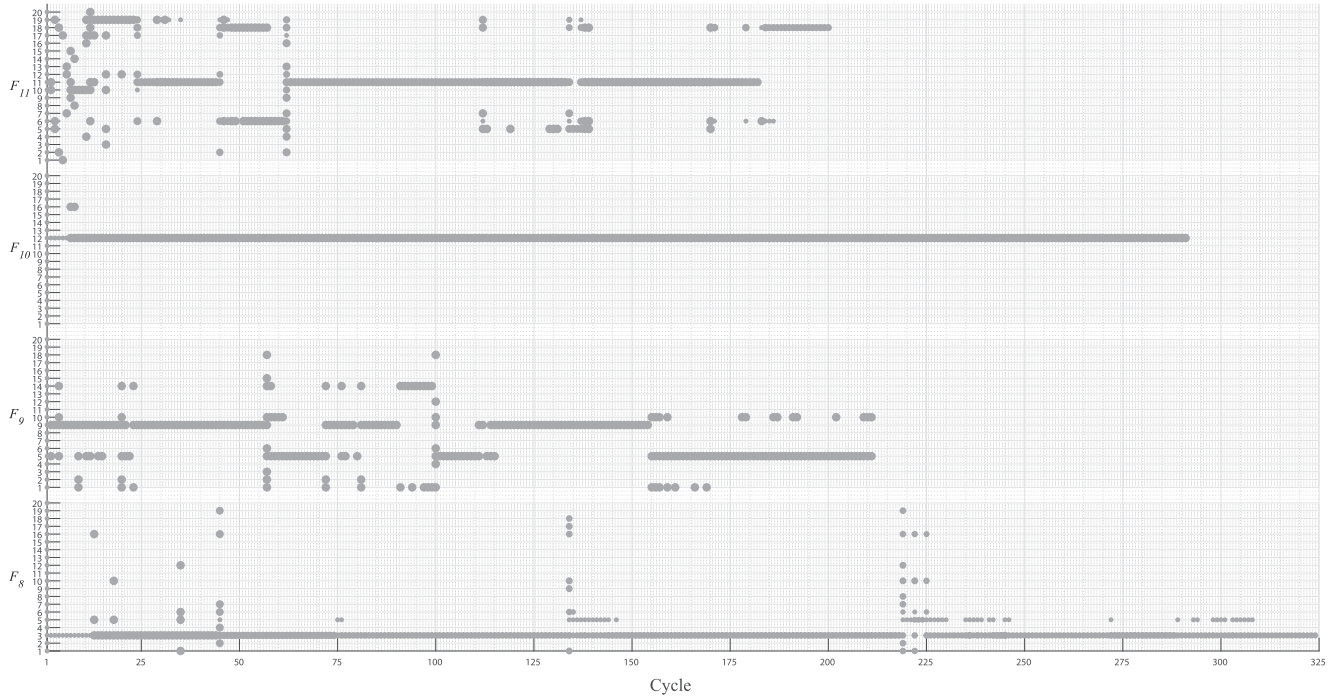


Fig. 8. Dynamic computational resource allocation scheme of DCCC on the partially separable functions F_8 – F_{11} with 20 components of CEC'2013 benchmark suite. The horizontal coordinate indicates the index of iterations and the vertical coordinate indicates 20 components of the functions. The gray circle indicates that the component is selected for optimization in this iteration, and the size of the circle indicates the allocated computational resources.

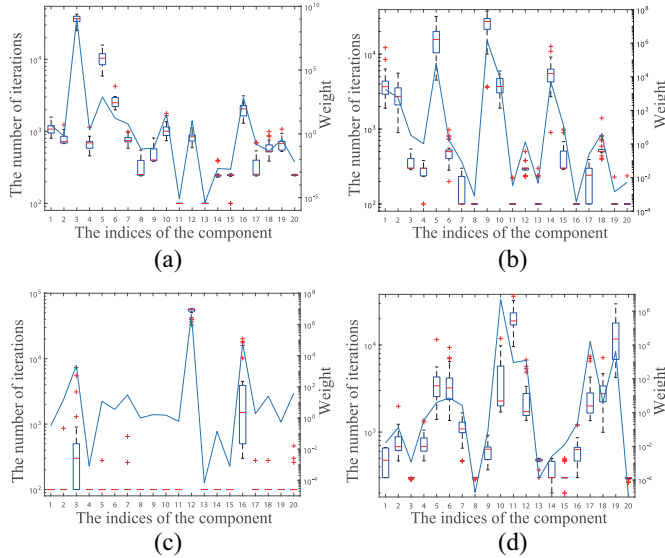


Fig. 9. Computational resource allocation scheme of DCCC on the partially separable functions F_8 – F_{11} with 20 components of CEC'2013 benchmark suite. The left ordinate represents the number of iterations allocated to the component, corresponding to the box plot, and the right ordinate represents the weighting of components by CEC'2013 benchmark, corresponding to the line plot. (a) F_8 . (b) F_9 . (c) F_{10} . (d) F_{11} .

LSOPs in the CEC'2013 benchmark suite as well as two heterogeneous LSOPs designed in Section V-A. It can be seen that DCCC achieves the best results on the eight functions, and the comparison results between DCCC and two ablation experiments are “6/3/4” and “8/3/2,” respectively. Therefore, the experimental results demonstrate that DCCC using two factors r and p_s to estimate the subproblem difficulty is superior

to DCCC using only one factor r or p_s . Furthermore, the results of two ablation experiments indicate that DCCC using only factor r could achieve better results on homogeneous LSOPs, while it was worse than DCCC using only factor p_s on both heterogeneous LSOPs problems. DCCC with only factor r could achieve positive results on homogeneous LSOPs because the search behaviors of the optimizers are similar for the same class of problems. On the other hand, the factor p_s could capture the differences in the search behaviors of the optimization algorithm dealing with different types of components and thus estimate the component difficulty more accurately.

Taking DCCC solving H_2 as an example, Fig. 10 shows the difficulty d , correlation coefficient r , and success probability p_s of the four key components 1 (f_8), 2 (f_2), 6 (f_{14}), and 11 (f_{11}) after each optimization component. In the early stages of optimization, it can be seen that $d(C1) \approx d(C6) \approx d(C11) > d(C2)$, $r(C2) > r(C11) > r(C6) > r(C1)$, and $p_s(C2) > p_s(C1) > p_s(C6) \approx p_s(C11)$. That is, components 1, 6, and 11 are significantly more difficult than component 2. This is consistent with the consensus that multimodal functions are more difficult than unimodal functions. Precisely because of the difference in components difficulty considered, DCCC allocated more computational resources to components 1, 6, and 11, and consequently achieved better results, as shown in Figs. 4 and 5. Except for component 2, the fitness landscape of component 1 is more difficult, while it is easily solved by SaNSDE. The experimental results in Table V demonstrate that it is more beneficial for DCCC to consider the problem landscape and algorithm behaviors together when estimating the component difficulty.

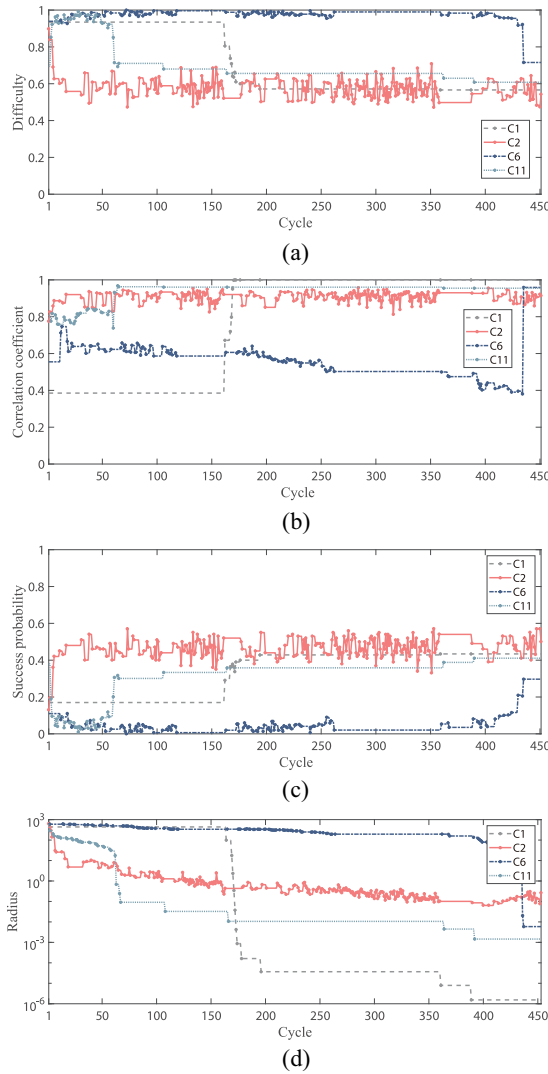


Fig. 10. Difficulty, correlation coefficient, success probability, and subpopulation radius of components 1, 2, 6, and 11 after each iteration in the process of DCCC solving H_2 . The marker point indicates that the component was optimized in this iteration. (a) Difficulty. (b) Correlation coefficient. (c) Success probability. (d) Subpopulation radius.

Moreover, Fig. 10(d) shows the trends of the subpopulation radius corresponding to four components to display the convergence of the subpopulations, where the subpopulation radius represents the distance of the subpopulation centroid from the outermost individuals of the subpopulation. The trends in Fig. 10 show that the difficulty of components 1, 6, and 11 plummets as the subpopulations tend to converge, especially as the correlation coefficient is close to 1, while the change in the probability of success is smaller. The experimental results show that the FDC coefficient could identify convergent subpopulations, and thus DCCC reduces the allocation of computational resources for the corresponding components to avoid wasting computational resources.

Finally, it is necessary to discuss the limitations of FDC. As a distance-based measure, the accuracy of FDC inevitably decreases as the dimensionality of the subproblem increases. Therefore, if the dimensionality of the components is too large, then it may not be reliable to use FDC values to estimate

the difficulty of the fitness landscape. However, in the experiments of this article, FDC has a positive effect on DCCC. The reason for this is that DCCC is based on the assumption that if there is a significant difficulty difference between components, then difficult components may be neglected by the contribution-aware CC framework because of their small early contributions. Therefore, the requirement of DCCC for difficulty estimation methods is to be able to identify simple and difficult components and to allocate more computational resources to difficult components based on their difficulty gap. Experimental results show that FDC could distinguish the simple components (e.g., unimodal function, component 2) from the difficult components (e.g., multimodal functions, components 1, 6, and 11) with a certain degree of accuracy. This rough estimate could assist DCCC in achieving the expected results, i.e., allocating more computational resources to difficult components to exploit their potential. Furthermore, the benchmarks used and constructed in this article have assumptions that are favorable to the employment of FDC. First, all benchmark functions assume that the dimension of each component is less than or equal to 100 dimensions. Second, two heterogeneous LSOPs assume significant difficulty differences between components.

VI. CONCLUSION

In this article, we have proposed a difficulty and contribution-based CC framework, called DCCC, which comprehensively considers the imbalance in the difficulty and contribution of the components in large-scale optimization. First, we have proposed a method to dynamically evaluate the problem difficulty during the evolution process. Then, we have designed a novel computational resource allocation scheme for the CC framework, where more difficult and more contributing components are allocated to more computational resources. Finally, we have tested the DCCC framework on CEC'2010 and CEC'2013 benchmarks, as well as two designed heterogeneous LSOPs, and have compared it with several CC frameworks and several non-CC large-scale optimization algorithms. The experimental results have demonstrated that DCCC is competitive in coping with separable and imbalanced LSOPs.

In future work, we will further study the imbalance feature in the large-scale optimization, and design a large-scale continuous optimization benchmark suite with heterogeneous components to express the distinction of the discipline context between the modules in real-world large-scale engineering design [1]. We will analyze the allocation of available computational resources on the large-scale optimization with imbalanced components, and the relationship between the investment and revenue on different types of components. Moreover, we will further study how to use the behavior of the optimization algorithm to more accurately estimate the difficulty of the problem, especially for high-dimensional problems.

ACKNOWLEDGMENT

The authors would like to thank the author Mohammad Nabi Omidvar from the University of Leeds for providing the

source code of CBCC3, and Zhenyu Yang for providing the source code of SaNSDE. The source code of DCCC could be available on Github: <https://github.com/sqrriler-xu/DCCC>.

REFERENCES

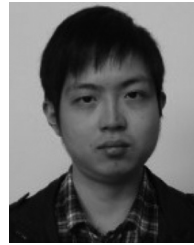
- [1] J. T. Hwang, A. V. Jain, and T. H. Ha, "Large-scale multidisciplinary design optimization—Review and recommendations," in *Proc. AIAA Aviation Forum*, 2019, p. 3106.
- [2] S. Cheng, B. Liu, Y. Shi, Y. Jin, and B. Li, "Evolutionary computation and big data: Key challenges and future directions," in *Proc. Int. Conf. Data Min. Big Data*, 2016, pp. 3–14.
- [3] B. Abbasi, T. Babaei, Z. Hosseinfard, K. Smith-Miles, and M. Dehghani, "Predicting solutions of large-scale optimization problems via machine learning: A case study in blood supply chain management," *Comput. Oper. Res.*, vol. 119, Jul. 2020, Art. no. 104941.
- [4] M. Pant, H. Zaheer, L. Garcia-Hernandez, and A. Abraham, "Differential evolution: A review of more than two decades of research," *Eng. Appl. Artif. Intell.*, vol. 90, Apr. 2020, Art. no. 103479.
- [5] X. Lin, W. Luo, and P. Xu, "Differential evolution for multimodal optimization with species by nearest-better clustering," *IEEE Trans. Cybern.*, vol. 51, no. 2, pp. 970–983, Feb. 2021.
- [6] Z. Li, X. Lin, Q. Zhang, and H. Liu, "Evolution strategies for continuous optimization: A survey of the state-of-the-art," *Swarm Evol. Comput.*, vol. 56, Aug. 2020, Art. no. 100694.
- [7] P. Xu, W. Luo, J. Xu, Y. Qiao, J. Zhang, and N. Gu, "An alternative way of evolutionary multimodal optimization: Density-based population initialization strategy," *Swarm Evol. Comput.*, vol. 67, Dec. 2021, Art. no. 100971.
- [8] M. R. Bonyadi and Z. Michalewicz, "Particle swarm optimization for single objective continuous space problems: A review," *Evol. Comput.*, vol. 25, no. 1, pp. 1–54, 2017.
- [9] C. Bu, W. Luo, and L. Yue, "Continuous dynamic constrained optimization with ensemble of locating and tracking feasible regions strategies," *IEEE Trans. Evol. Comput.*, vol. 21, no. 1, pp. 14–33, Feb. 2017.
- [10] M. N. Omidvar, X. Li, and K. Tang, "Designing benchmark problems for large-scale continuous optimization," *Inf. Sci.*, vol. 316, pp. 419–436, Sep. 2015.
- [11] M. A. Potter and K. A. De Jong, "A cooperative coevolutionary approach to function optimization," in *Proc. Int. Conf. Parallel Problem Solving Nature*, 1994, pp. 249–257.
- [12] P. Xu, W. Luo, X. Lin, J. Zhang, Y. Qiao, and X. Wang, "Constraint-objective cooperative coevolution for large-scale constrained optimization," *ACM Trans. Evol. Learn. Optim.*, vol. 1, no. 3, pp. 1–26, 2021.
- [13] W. Luo, B. Yang, C. Bu, and X. Lin, "A hybrid particle swarm optimization for high-dimensional dynamic optimization," in *Simulated Evolution Learning*. Cham, Switzerland: Springer, 2017, pp. 981–993.
- [14] P. Yang, K. Tang, and X. Yao, "Turning high-dimensional optimization into computationally expensive optimization," *IEEE Trans. Evol. Comput.*, vol. 22, no. 1, pp. 143–156, Feb. 2018.
- [15] V. D. B. Frans and A. P. Engelbrecht, "A cooperative approach to particle swarm optimization," *IEEE Trans. Evol. Comput.*, vol. 8, no. 3, pp. 225–239, Jun. 2004.
- [16] Z. Yang, K. Tang, and X. Yao, "Differential evolution for high-dimensional function optimization," in *Proc. IEEE Congr. Evol. Comput.*, 2007, pp. 3523–3530.
- [17] X. Li and X. Yao, "Cooperatively coevolving particle swarms for large scale optimization," *IEEE Trans. Evol. Comput.*, vol. 16, no. 2, pp. 210–224, Apr. 2012.
- [18] M. N. Omidvar, X. Li, Y. Mei, and X. Yao, "Cooperative co-evolution with differential grouping for large scale optimization," *IEEE Trans. Evol. Comput.*, vol. 18, no. 3, pp. 378–393, Jun. 2014.
- [19] M. N. Omidvar, M. Yang, Y. Mei, X. Li, and X. Yao, "DG2: A faster and more accurate differential grouping for large-scale black-box optimization," *IEEE Trans. Evol. Comput.*, vol. 21, no. 6, pp. 929–942, Dec. 2017.
- [20] Y. Sun, M. Kirley, and S. K. Halgamuge, "A recursive decomposition method for large scale continuous optimization," *IEEE Trans. Evol. Comput.*, vol. 22, no. 5, pp. 647–661, Oct. 2018.
- [21] Y. Sun, M. N. Omidvar, M. Kirley, and X. Li, "Adaptive threshold parameter estimation with recursive differential grouping for problem decomposition," in *Proc. Genet. Evol. Comput. Conf.*, 2018, pp. 889–896.
- [22] M. N. Omidvar, X. Li, and X. Yao, "Smart use of computational resources based on contribution for cooperative co-evolutionary algorithms," in *Proc. 13th Annu. Conf. Genet. Evol. Comput.*, 2011, pp. 1115–1122.
- [23] M. N. Omidvar, B. Kazimipour, X. Li, and X. Yao, "CBCC3—A contribution-based cooperative co-evolutionary algorithm with improved exploration/exploitation balance," in *Proc. IEEE Congr. Evol. Comput.*, 2016, pp. 3541–3548.
- [24] M. Yang *et al.*, "Efficient resource allocation in cooperative co-evolution for large-scale global optimization," *IEEE Trans. Evol. Comput.*, vol. 21, no. 4, pp. 493–505, Aug. 2017.
- [25] Z. Ren, Y. Liang, A. Zhang, Y. Yang, Z. Feng, and L. Wang, "Boosting cooperative coevolution for large scale optimization with a fine-grained computation resource allocation strategy," *IEEE Trans. Cybern.*, vol. 49, no. 12, pp. 4180–4193, Dec. 2019.
- [26] Y.-H. Jia *et al.*, "Distributed cooperative co-evolution with adaptive computing resource allocation for large scale optimization," *IEEE Trans. Evol. Comput.*, vol. 23, no. 2, pp. 188–202, Apr. 2019.
- [27] B. Kazimipour, M. N. Omidvar, A. K. Qin, X. Li, and X. Yao, "Bandit-based cooperative coevolution for tackling contribution imbalance in large-scale optimization problems," *Appl. Soft Comput.*, vol. 76, pp. 265–281, Mar. 2019.
- [28] W. Luo, Y. Qiao, X. Lin, P. Xu, and M. Preuss, "Many-modal optimization by difficulty-based cooperative co-evolution," in *Proc. IEEE Symp. Comput. Intell.*, 2019, pp. 1907–1914.
- [29] T. Jones and S. Forrest, "Fitness distance correlation as a measure of problem difficulty for genetic algorithms," in *Proc. 6th Int. Conf. Genet. Algorithms*, vol. 95, 1995, pp. 184–192.
- [30] X. Li, K. Tang, M. N. Omidvar, Z. Yang, and K. Qin, "Benchmark functions for the CEC'2013 special session and competition on large-scale global optimization," School Comput. Sci. Inf. Technol., RMIT Univ., Melbourne, VIC, Australia, Rep., 2013. [Online]. Available: <https://titian.csit.rmit.edu.au/~e46507/publications/lsgo-cec10.pdf>
- [31] X. Ma *et al.*, "A survey on cooperative co-evolutionary algorithms," *IEEE Trans. Evol. Comput.*, vol. 23, no. 3, pp. 421–441, Jun. 2019.
- [32] Z. Tan, K. Li, and Y. Wang, "Differential evolution with adaptive mutation strategy based on fitness landscape analysis," *Inf. Sci.*, vol. 549, pp. 142–163, Mar. 2021.
- [33] X. Chu, F. Cai, J. Chen, and L. Li, "Recommending PSO variants using meta-learning framework for global optimization," in *Proc. Asia-Pacific Conf. Simulat. Evol. Learn.*, 2017, pp. 939–948.
- [34] W. Li, X. Meng, and Y. Huang, "Fitness distance correlation and mixed search strategy for differential evolution," *Neurocomputing*, vol. 458, pp. 514–525, Oct. 2021.
- [35] K. Sallam, S. Elsayed, R. Sarker, and D. Essam, "Landscape-based differential evolution for constrained optimization problems," in *Proc. IEEE Congr. Evol. Comput.*, 2018, pp. 1–8.
- [36] R. Storm and K. Price, "Differential evolution—A simple and efficient heuristic for global optimization over continuous spaces," *J. Global Optim.*, vol. 11, no. 4, pp. 341–359, 1997.
- [37] P. Kerschke, M. Preuss, S. Wessing, and H. Trautmann, "Low-budget exploratory landscape analysis on multiple peaks models," in *Proc. Genet. Evol. Comput. Conf.*, 2016, pp. 229–236.
- [38] O. Mersmann, B. Bischl, H. Trautmann, M. Preuss, C. Weihs, and G. Rudolph, "Exploratory landscape analysis," in *Proc. 13th Annu. Conf. Genet. Evol. Comput.*, 2011, pp. 829–836.
- [39] M. Schumer and K. Steiglitz, "Adaptive step size random search," *IEEE Trans. Autom. Control*, vol. 13, no. 3, pp. 270–276, Jun. 1968.
- [40] K. Tang, X. Li, P. N. Suganthan, Z. Yang, and T. Weise, "Benchmark functions for the CEC'2010 special session and competition on large-scale global optimization," Nat. Inspired Comput. Appl. Lab., School Comput. Sci. Technol., Univ. Sci. Technol. China, Hefei, China, Rep., 2010. [Online]. Available: https://al-roomi.org/multimedia/CEC_Database/CEC2015/LargeScaleGlobalOptimization/CEC2015_LargeScaleGO_TechnicalReport.pdf
- [41] Z. Yang, K. Tang, and X. Yao, "Self-adaptive differential evolution with neighborhood search," in *Proc. Congr. Evol. Comput.*, 2008, pp. 1110–1116.
- [42] R. Tanabe and A. Fukunaga, "Success-history based parameter adaptation for differential evolution," in *Proc. IEEE Congr. Evol. Comput.*, 2013, pp. 71–78.
- [43] D. Molina, M. Lozano, and F. Herrera, "MA-SW-chains: Memetic algorithm based on local search chains for large scale continuous global optimization," in *Proc. IEEE Congr. Evol. Comput.*, 2010, pp. 1–8.
- [44] A. LaTorre, S. Muelas, and J.-M. Peña, "Large scale global optimization: Experimental results with MOS-based hybrid algorithms," in *Proc. IEEE Congr. Evol. Comput.*, 2013, pp. 2742–2749.

- [45] A. A. Hadi, A. W. Mohamed, and K. M. Jambi, "LSHADE-SPA memetic framework for solving large-scale optimization problems," *Complex Intell. Syst.*, vol. 5, no. 1, pp. 25–40, 2019.
- [46] Q. Yang, W.-N. Chen, J. Da Deng, Y. Li, T. Gu, and J. Zhang, "A level-based learning swarm optimizer for large-scale optimization," *IEEE Trans. Evol. Comput.*, vol. 22, no. 4, pp. 578–594, Aug. 2018.
- [47] D. Li, W. Guo, A. Lerch, Y. Li, L. Wang, and Q. Wu, "An adaptive particle swarm optimizer with decoupled exploration and exploitation for large scale optimization," *Swarm Evol. Comput.*, vol. 60, Feb. 2021, Art. no. 100789.
- [48] J. J. Liang, B. Y. Qu, P. N. Suganthan, and A. G. Hernández-Díaz, "Problem definitions and evaluation criteria for the CEC 2013 special session on real-parameter optimization," *Comput. Intell. Lab., Zhengzhou Univ., Zhengzhou, China, Rep.* 201212, 2013.



Peilan Xu received the B.S. degree from Dalian Polytechnic University, Dalian, China, in 2017. He is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, University of Science and Technology of China, Hefei, China.

He is currently a visiting student with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China. His research interests include large-scale global optimization algorithms and applications.



Xin Lin received the B.E. degree from Wuhan Textile University, Hubei, China, in 2013. He is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, University of Science and Technology of China, Hefei, China.

He is currently a visiting student with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China. His research interests include evolutionary optimization and applications.



Yatong Chang (Graduate Student Member, IEEE) received the B.S. degree from Beihang University, Beijing, China, in 2016. He is currently pursuing the master's degree with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China.

His research interests include evolutionary optimization and applications.



Wenjian Luo (Senior Member, IEEE) received the B.S. and Ph.D. degrees from the Department of Computer Science and Technology, University of Science and Technology of China, Hefei, China, in 1998 and 2003, respectively.

He is currently a Professor with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China. His current research interests include computational intelligence and applications, network security and data privacy, machine learning, and data mining.

Prof. Luo currently serves as an Associate Editor or editorial board member for several journals, including *Information Sciences*, *Swarm and Evolutionary Computation*, *Journal of Information Security and Applications*, *Applied Soft Computing*, and *Complex & Intelligent Systems*. He also serves as the Chair of the IEEE CIS ECTC Task Force on Artificial Immune Systems. He has been a member of the Organizational Team of more than ten academic conferences, in various functions, such as a program chair, a symposium chair, and a publicity chair. He is a Distinguished Member of CCF and a Senior Member of ACM and CAAI.



Ke Tang (Senior Member, IEEE) received the B.Eng. degree from the Huazhong University of Science and Technology, Wuhan, China, in 2002, and the Ph.D. degree from Nanyang Technological University, Singapore, in 2007.

From 2007 to 2017, he was with the School of Computer Science and Technology, University of Science and Technology of China, Hefei, China, first as an Associate Professor from 2007 to 2011 and later a Professor from 2011 to 2017. He is currently a Professor with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China. He has published more than 70 journal papers and more than 80 conference papers. According to Google Scholar, his publications have received more than 8000 citations and the H-index is 42. His major research interests include evolutionary computation, machine learning, and their applications.